

CORE+: A Complex Event Recognition Engine in C++

Kyle Bossonney
kyle@bossonney.com
Oxford University
Oxford, United Kingdom

Nicolás Buzeta
nicolas.buzeta@uc.cl
PUC Chile & IMFD Chile
Santiago, Chile

Vicente Calisto
vecalisto@uc.cl
PUC Chile & IMFD Chile
Santiago, Chile

Juan-Eduardo López
juan.lpez@uc.cl
PUC Chile & IMFD Chile
Santiago, Chile

Cristian Riveros
cristian.riveros@uc.cl
PUC Chile & IMFD Chile
Santiago, Chile

Stijn Vansummeren
stijn.vansummeren@uhasselt.be
UHasselt, Data Science Institute
Diepenbeek, Belgium

Abstract

Complex Event Recognition (CER) refers to the activity of analyzing streams of continuously arriving event data, to recognize collections of events that satisfy user-defined patterns. CER is known to be computationally challenging, since it requires maintaining a set of partial matches, and this set quickly grows super-linearly in the number of processed events. In this paper we demonstrate CORE+, a mature COMplex event Recognition Engine that circumvents the super-linear partial match problem. CORE+ is based on a research prototype named CORE; we demonstrate the new features and capabilities that make CORE+ a release version of CORE.

CCS Concepts

• **Information systems** → **Stream management**; *Data streams*.

Keywords

Data streaming, complex event recognition.

ACM Reference Format:

Kyle Bossonney, Nicolás Buzeta, Vicente Calisto, Juan-Eduardo López, Cristian Riveros, and Stijn Vansummeren. 2025. CORE+: A Complex Event Recognition Engine in C++. In *Companion of the 2025 International Conference on Management of Data (SIGMOD-Companion '25)*, June 22–27, 2025, Berlin, Germany. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3722212.3725090>

1 Introduction

Complex Event Recognition (CER) is a prominent technology for modern streaming applications in domains such as maritime surveillance, cybersecurity, industrial automation, and real-time decision-making [6]. CER systems analyze high-speed streams of raw events to detect *complex events*: collections of individual events that conform to a specified pattern. These patterns are defined through expressive queries that match incoming events on the basis of their content, their temporal order, and their relationship with previous

events. By identifying complex events, CER systems provide time-sensitive insights and allow reactive response to critical situations.

Given the importance of CER, numerous CER query languages and systems have been proposed over the past decades, see, e.g., [6] for a recent survey of the field. Unfortunately, it has been repeatedly observed [2, 6, 7] that while these proposals have a common goal, they differ vastly in their architectures, data models, pattern languages, and processing mechanisms. Furthermore, these systems suffer from two crucial deficiencies: (1) they often lack precise syntax and semantics of their pattern language, making it difficult to compare to other systems, and (2) the proposed processing algorithms offer no performance guarantees, and they do not scale on often surprisingly simple patterns.

Towards resolving these shortcomings, a subset of the current authors have developed CEQL [2], a complex event query language with a formally defined syntax and semantics. CEQL contains all basic operators common to most CER languages, and its expressiveness is by now well understood [7]. Additionally, we have developed a processing algorithm that has *strong performance guarantees*: it takes constant time to process each new event (regardless of how many events have been received before) and, whenever new complex events $C_1, \dots, C_n$ are recognized, these can be retrieved them in time that is linear in the size of the next C_i and independent of how many events were previously received or recognized.

A prototype system named CORE, which implemented CEQL and its evaluation algorithm, was presented in [2], with a focus on explaining the algorithmic framework behind the system and demonstrating the scalability of the processing algorithm. In this demonstration we show a release version named CORE+. This new version incorporates the following new features and capabilities:

- (1) CORE was implemented in Java whereas CORE+ is implemented in C++. This makes CORE+ in general more efficient. In particular, whereas CORE+ depended on the Java garbage collector, which could be called at unpredictable times, CORE+ explicitly manages its own memory in a predictable and optimized fashion.
- (2) CORE followed a monolithic, centralized, and single-threaded architecture. By contrast, CORE+ has a client-server architecture that allows distributed processing, and natively processes each query in a separate thread.
- (3) CORE+ provides a Python library for clients and streamers, namely, the middle-ware software to connect real data streams to CORE+. Further, CORE was restricted to processing events in arrival order.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD-Companion '25, Berlin, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1564-8/2025/06
<https://doi.org/10.1145/3722212.3725090>

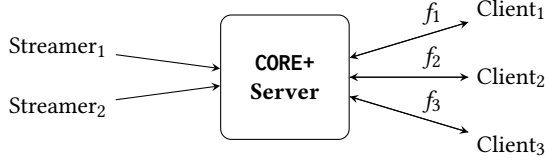


Figure 1: CORE+ publish-subscribe software architecture.

CORE+ adds the ability to process events based on a user-specified order, and has a *quarantine* module to deal with out-of-order events.

(4) CORE+ implements a more general query language featuring new CEQL operators, such as contiguous sequencing, contiguous iteration, and conjunction, among others.

(5) CORE+ has a web interface that allows to try and test CEQL queries over data streams in real time.

In this demo, we will showcase the capabilities and inherent scalability of CORE+ by means of the real-world use case of querying the data streams generated by a cryptocurrency exchange platform called Coinbase [3]. The CORE+ code is publicly available at [4] and the web interface at <https://core.ing.uc.cl/>.

2 Data stream processing in CORE+

The CORE+ system serves as a database server designed for the efficient querying of event streams. Positioned in-between the event streams and the clients (i.e., users), the system ingests streaming data, processes client queries, and delivers complex events in response. Clients interact with the server by registering queries and subscribing to results, which are continuously produced. Figure 1 illustrates the software architecture of CORE+, discussed next.

CORE+ ingests streaming data by means of so-called *streamers*: middleware software that connect to third-party data sources (e.g., using WebSocket or a message-queue specific protocol) and that transmit the streaming source data to the CORE+ server in a standardized format. CORE+ includes a Python API for developing streamers. This API offers the methods necessary to connect to the CORE+ server, and to send data in the required format. While the API is currently available only in Python, plans are in place to extend support to other popular programming languages. Streamers transmit data to the CORE+ server over a TCP connection and can therefore run either on the same machine as the CORE+ server, or on a separate one. Figure 1 (left) illustrates an example with two streamers.

To query data streams, clients can register queries written in CEQL (CORE+’s query language, detailed in Section 3) at the server. For this purpose, CORE+ offers a Python client library that follows a publish-subscribe pattern. Using this library, a client can submit a query Q to the server, together with a callback function f . Instead, if Q was already created (e.g., by another client), the client can subscribe to Q by just passing f . Whenever the server recognizes a new match of Q in the data stream, it generates the associated set S of complex events and invokes all callback functions f of clients subscribed to Q . Each client can then read S and act accordingly. Figure 1 (right) illustrates the interaction between clients subscribing to queries and the server publishing the results.

CORE+ models streams as sequences of events, where each event is viewed as a relational record $T(a_1: v_1, \dots, a_n: v_n)$ that has a name T , attributes a_i , and associated values v_i . The record name

T is called the *event type* and all events of type T must have the same set of attributes $a_1, \dots, a_n$. CORE+ provides a *data description language* (DDL) that is used by streamers to declare the *schema* of their stream to the CORE+ server. This schema specifies the event types that can occur, their attributes, and the types that the attribute values should take. The DDL has the following general syntax.

```
CREATE STREAM < stream-name > {
  EVENT < event-type-name > {
    < attribute-name > : < data-type >, ...
  },
  EVENT < event-type-name > { ... }, ...
}
```

Here stream-name and the event-type-names must be unique identifiers that can be used refer to the stream and its events in queries. Currently, CORE+ supports four data types: int, string, double, and boolean. We plan to support more data types (e.g., date-time) for the CORE+ demonstration.

The order of events is crucial for querying in complex event recognition, as users often care about the temporal sequence in which events occur—whether based on when the events were created or when they arrive in the system. The CORE+ query language includes specialized operators (e.g., the sequencing operator $\circ$) to query this order. By default, CORE+ uses the same order as the order in which events occur in the source data stream. However, users may also indicate that they wish to reorder events. In that case, they must specify the event attribute that determines the new order of events in the stream by assigning this attribute the *primary_time* datatype in the CREATE STREAM statement. For consistency, if one event type of a stream uses the *primary_time* datatype, then all event types of the stream must do so.

One issue that arises when events are to be reordered by means of the *primary_time* feature is that in principle one needs to see the entire original stream in order to produce the correctly reordered stream on which patterns will be recognized. This requirement conflicts with the desire to process the input streams incrementally, without ever seeing them completely. Therefore, an incremental way for reordering is necessary. For this purpose, CORE+’s DDL language also allows users to declare a *quarantine* for each stream:

```
CREATE QUARANTINE {
  < policy > [< window-size >] { < stream-name > }
}
```

The *policy* is the algorithm that will be used to perform the quarantine. Currently, CORE+ supports two quarantine policies: DIRECT and FIXED_TIME. DIRECT defines no quarantine i.e., events pass directly to the server. FIXED_TIME defines that each event must stay in quarantine a fixed amount of (wall-clock) time after it is ingested and waiting for “late events” to arrive. Those are events that arrive later but that have a smaller *primary_time* value. Once the quarantine time is expired the event, and all the recognized late events, are sent to the server. Further late events that arrive after this are discarded. We plan to support more quarantine policies during the demonstration. Note that one can see quarantine as a restricted form of *watermarking* [1] applied only when the stream arrives to ensure no out-of-order events in the stream.

Demo use case. During the demonstration, we will show CORE+ working with real data streams generated by a cryptocurrency exchange platform called Coinbase, one of the largest cryptocurrency

exchanges in the USA regarding trading volume. On this platform, users can buy or sell cryptocurrencies like Bitcoins and Ethereum, among hundreds of other assets. Each user can place a bid price if they want to buy some volume of a cryptocurrency or ask for a price if they want to sell. If a bid matches the asked price, the system automatically exchanges it between users. Coinbase's streams report all the events generated during the exchange in real time and are publicly available by a WebSocket API [3].

The demonstration will include most WebSocket channels available as data streams. For instance, the *ticker channel* provides real time price updates every time a match happens. In CORE+, we model this channel as a stream of buy and sell events, defined as:

```
CREATE STREAM TICKER {
  EVENT Buy {
    product_id : string,    price : double,
    open_24h : double,      volume_24h : double,
    low_24h : double,        high_24h : double,
    volume_30d : double,     best_bid : double,
    best_bid_size : double,  best_ask : double,
    best_ask_size : double,  last_size : double,
    time : primary_time
  },
  EVENT Sell {
    (same attributes as Buy)
  }
}
```

where `product_id` is the type of the cryptocurrency (e.g., Ethereum), `price` is the price given in US dollars, etc. The match is a buy or a sell, depending on whether the bid or the asked price was made first.

Each Coinbase stream produces events at a rate of more than 30 events per second (around 10 KB per second). Their streams are relatively stable, and events are correctly ordered by the time attribute (i.e. when the event happens). For security and demonstration purposes, we will put a quarantine over each stream of five seconds (i.e., `FIXED_TIME`). Each WebSocket channel will correspond to a stream whose data schema will be available at the Web interface (see Section 5). Further, as explained in the next section, CORE+'s users will be able to pose CEQL queries to these streams in real time during the demo.

3 CEQL: Syntax and features

The Complex Event Query Language (CEQL), originally introduced in [2], is a declarative language for specifying complex event patterns. CORE+ fully implements CEQL and additionally extends the language with new operators and clauses. In this section, we review the basic CEQL syntax, discuss the added operators, and provide examples that will be utilized during the demonstration.

Syntactically, a CEQL query has the following form:

```
SELECT      < list-of-variables >
FROM        < list-of-streams >
WHERE       < stream-pattern >
[PARTITION BY < list-of-attributes >]
[WITHIN     < time-value >]
[CONSUME BY < consumption-policy >]
[LIMIT      < number-of-outputs >]
```

The `FROM` clause specifies the list of streams that will be queried. The optional `PARTITION BY` clause allows to partition the streams on a specific set of attributes, so that matches can be recognized on

each partition in parallel. The `WHERE` clause specifies the pattern to recognize in the stream. This pattern is specified using the following syntax, based on Complex Event Logic [7]:

<code>p</code>	<code>:=</code>	<code>R</code>	(event type selection, e.g., Buy)
		<code>p AS X</code>	(variable binding)
		<code>p FILTER X[P]</code>	(filtering)
		<code>p;p</code>	(non-contiguous sequencing)
		<code>p+</code>	(non-contiguous iteration)
		<code>p OR p</code>	(disjunction)
		<code>p:p</code>	(contiguous sequencing)
		<code>p:+</code>	(contiguous iteration)
		<code>p AND p</code>	(conjunction)
		<code>p ALL p</code>	(interleaved conjunction)

CEQL patterns `p` hence include the standard operators used in CER systems [5, 6]. A distinctive feature of CORE and CORE+ is that operators can freely be nested in `p`; competing systems do not allow this feature [7]. The last four patterns above are new operators of CORE+ that are not supported by CORE: contiguous sequencing and iteration (`p:p` and `p:+`), conjunction (`p AND p`), and interleaved conjunction (`p ALL p`). Furthermore, CORE+ also includes a new form of predicate in filters, `X[P]`, that allows regular expression matching on string values. We refer to [7] for a formal semantics of all operators.

A CEQL query's `WITHIN` clause specifies the time window within which the pattern must be recognized (e.g., within five minutes). The `SELECT` clause specifies the individual event variables that will be included in the output complex event.

CORE+ adds two new clauses to CORE: `CONSUME BY` and `LIMIT`. `CONSUME BY` allows to specify how events will be consumed when a match happens (cf, e.g. [5]). Currently, CORE+ supports three consumption policies: `ANY` which drops all events when a match happens; `PARTITION` which drops all events in the partition; and `NONE` (i.e., the default) that performs no consumption. The `LIMIT` clause allows to restrict the number of outputs, improving performance in the server and network bandwidth between the server and the clients.

Example queries over Coinbase. To see how to write queries in CEQL, we will show and run queries over the Coinbase stream during the demonstration. Given space restrictions, here we only present two use cases over the Ticker stream, whose schema was introduced in the previous section. The demonstration will include a more significant set of queries over different streams from Coinbase.

Example 3.1. To start, suppose that a user wants to raise an alert whenever there are three buy-events of bitcoins (BTC-USD) where its price is over some threshold (e.g., US\$100.000), which can be a notification for selling bitcoins. One can specify this as follows:

```
SELECT B3 FROM TICKER
WHERE Buy; Buy; (Buy AS B3)
FILTER Buy[product_id = 'BTC-USD' AND price > 100000]
WITHIN 30 SECONDS
LIMIT 1
```

The `WHERE` clause asks to see three buys (not necessarily contiguous), while the `FILTER` clause applies conditions to all three events at once. Further, we use variable binding called `B3` to only output the last event in the `SELECT` clause. Finally, we restrict the three

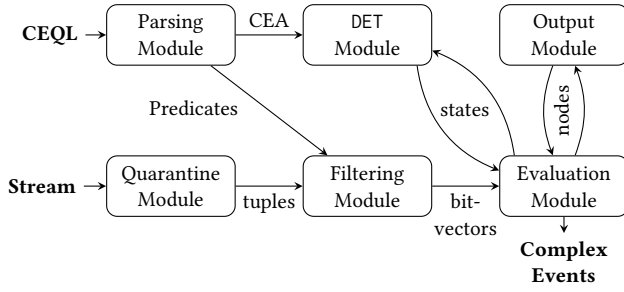


Figure 2: CORE+ internal architecture.

buy events to occur within a time window of 30 seconds and limit to one complex event per match.

Example 3.2. For a more selective query, suppose a user wants to extract all sell and buy events between two buy events that reach the lowest and highest price in 24 hours for the same cryptocurrency.

```

SELECT List FROM TICKER
WHERE Buy AS B1 : (Sell OR Buy):+ AS List : Buy AS B2
FILTER B1[price <= low_24h] AND B2[price >= high_24h]
PARTITION BY product_id
WITHIN 30 MINUTES

```

In this query, we use the PARTITION BY clause to require all events to have the same product_id. Contiguous sequencing (:) and contiguous iteration (:+) are used to specify that we want *all* the sell and buy events occurring between the lowest and highest buy, called B1 and B2, respectively. Then, we bind this list with the variable List and output it in the SELECT clause. The entire pattern must occur within 30 minutes.

4 CORE+ internal architecture

Figure 2 illustrates the internal processing architecture of CORE+. Each CEQL query is compiled by the Parsing module into a *Complex Event Automaton* (CEA for short) [2, 7], which is an extension of finite state automaton capable of recognizing complex events. CORE+'s evaluation algorithm (implemented in the *Evaluation Module*) is defined in terms of CEA: given a *deterministic* CEA (generated by the *DET module*) and a stream, it allows to process each new input event and check for complex event recognition. In [2] we showed that the evaluation algorithm has *optimal* complexity: each new input event is processed in constant time (regardless of the number of events already received) and, if a complex event C is recognized, then we can output C in time that is linear in the size of C (independent of how many events were previously received or recognized). The algorithm works achieves this by incrementally maintaining a graph-based data structure that compactly represents partials outputs. These data structures are managed by the *Output module*, and form the state that is maintained during matching.

The evaluation module does not operate directly on the input stream. Rather, the stream is first passed through the *Quarantine Module*, which implements the specified quarantine logic (see Section 2). Subsequently, the *Filtering Module* drops all events that does not satisfy the primitive filters specified in the CEQL query. For efficiency reasons, each event is transformed into a bit vector that indicates the set of predicates satisfied before being passing on to the Evaluation Module.

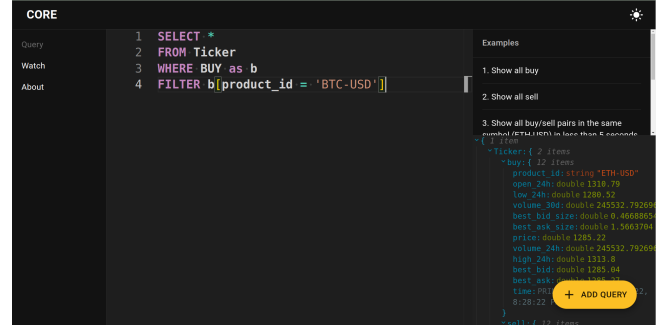


Figure 3: Screenshots of CORE+ web interface.

CORE, the prototype system described in [2], has a similar architecture. However, while CORE is implemented in Java and relies on the JVM to automatically collect garbage memory, CORE+ is implemented in C++ allowing for predictable and fine-grained memory management. Additionally, while CORE is single-threaded, CORE+ evaluates each query in a separate thread.

5 Web interface and demonstration

During the demo, the audience will access the CORE+ web interface running over the Coinbase stream in real-time. CORE+ is designed as a database server where clients can query and consume outputs by using the CORE+ Python library. The web interface is planned for testing CEQL queries and fits well for demonstration purposes.

In Figure 3, we display a screenshot of CORE+ web interface. The interface has a module for writing and subscribing queries, where users can also see the stream schemas and access example queries to aid the user experience. Users can see the results of their queries in real-time in the output module. There, the user can choose which queries to watch or delete if the query is unnecessary. The interface also includes other features for tracking the outputs, such as slowing the speed streams. The CORE+ code is publicly available at [4]. The audience can try the web interface at <https://core.ing.uc.cl/>.

Acknowledgments. This work was funded by ANID Fondecyt Regular project 1230935, and by ANID – Millennium Science Initiative Program – Code ICN17_002. S. Vansummeren was supported by the Research Foundation Flanders (FWO) under Grant No. G0B9623N and the UHasselt Bijzonder Onderzoeksfonds (BOF) under Grant No. BOF20ZAP02.

References

- [1] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. 2013. Millwheel: Fault-tolerant stream processing at internet scale. *VLDB* 6, 11 (2013).
- [2] Marco Bucchì, Alejandro Grez, Andrés Quintana, Cristian Riveros, and Stijn Vansummeren. 2022. CORE: a COmplex event Recognition Engine. *Proc. VLDB Endow.* 15, 9 (2022), 1951–1964.
- [3] Coinbase. 2025. Coinbase Websocket API. <https://docs.cdp.coinbase.com/exchange/docs/websocket-channels>.
- [4] CORE. 2025. CORE repository. <https://github.com/CORE-cer/CORE>.
- [5] Gianpaolo Cugola and Alessandro Margara. 2012. Processing flows of information: From data stream to complex event processing. *ACM Computing Surveys (CSUR)* 44, 3 (2012), 1–62.
- [6] Nikos Giatrakos, Elias Alevizos, Alexander Artikis, Antonios Deligiannakis, and Minos Garofalakis. 2020. Complex event recognition in the big data era: a survey. *The VLDB Journal* 29, 1 (2020), 313–352.
- [7] Alejandro Grez, Cristian Riveros, Martín Ugarte, and Stijn Vansummeren. 2021. A Formal Framework for Complex Event Recognition. *ACM Trans. Database Syst.* 46, 4 (2021), 16:1–16:49.